

PRACTICA

PRÁCTICA · UNIDAD III · WEB 3.0

DE LA FICHA AL DIPLOMA

Dos laboratorios encadenados: un token fungible y una credencial única. Todo en la red local de Hardhat, sin ETH.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

Hoy construimos lo que ayer explicamos. Dos laboratorios en parejas, en máquina, todo en la red local de Hardhat: sin ETH, sin red pública, sin IPFS.

A	**LAB 1 · TOKEN ERC-20 (FICHAUSB + CANJE)** Token con tope y quema, y un puesto que cobra con approve + transferFrom. Completar 4 TODO hasta 12 pruebas en verde.	~80 min
B	**LAB 2 · NFT (DIPLOMA + SBT)** DiplomaUSB (ERC-721 con tope) y el intransferible DiplomaSBT (ERC-5192). Hasta 20 pruebas en verde.	~70 min

EL PROPÓSITO DE LA SESIÓN

AL SALIR DE AQUÍ

Completar los contratos hasta que las pruebas pasen (12 en el Lab 1, 20 en el Lab 2), entendiendo cada pieza.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

01 ¿Cómo se COMPONE un token a partir de las extensiones de OpenZeppelin?

02 ¿Por qué approve + transferFrom mueve mis tokens sin entregar mi clave?

03 ¿Qué hace único a un NFT, y dónde vive de verdad su metadata?

04 ¿Qué cambia una sola línea en _update para volver intransferible un diploma?

RA3 · Programar contratos aplicando estándares (ERC-20, ERC-721, OpenZeppelin) · RA4 · Suite de pruebas automatizadas.

CÓMO SE TRABAJA HOY

Trabajan sobre el ANDAMIAJE: los contratos con TODO que ya compilan. Copian cada archivo sobre contracts/, completan los TODO en orden y corren las pruebas hasta verlas en verde. La solución no se entrega: es justo el objetivo.

EN LA CARPETA DEL LABORATORIO

```
npm install                                                    # una sola vez
npx hardhat test test/s10/FichaUSB.test.js                     # Lab 1 · meta: 12 passing
npx hardhat test test/s11/DiplomaUSB.test.js test/s11/DiplomaSBT.test.js   # Lab 2 · meta: 20 passing
```

DOS RECORDATORIOS

npx hardhat test recibe ARCHIVOS, no carpetas. Y todo corre en la red local de Hardhat: sin ETH, sin red pública y sin subir nada a IPFS. El tokenURI del NFT será una cadena de prueba.

El orden sugerido: terminen el Lab 1 completo antes de abrir el Lab 2. El intransferible (DiplomaSBT) es un reto opcional que depende de que DiplomaUSB ya esté listo.

A

APROXIMADAMENTE 80 MINUTOS · EN PAREJAS

LAB 1 · TOKEN ERC-20

FichaUSB (token con tope y quema) y Canje (el patrón approve + transferFrom). Cuatro TODO, doce pruebas en verde.

FICHAUSB Y CANJE

PARTE	QUÉ HACE	ESTÁNDAR QUE APLICA
FichaUSB	Token de puntos con tope, acuñación controlada y quema.	ERC20 + Capped + Burnable + Ownable
acunar	El organizador crea puntos, hasta el tope.	_mint con control de acceso
Canje	Un puesto que cobra fichas por un premio, usando la aprobación.	approve + transferFrom

```
# Se copia el andamiaje sobre contracts/s10 y se completan los TODO 1-4
npx hardhat test test/s10/FichaUSB.test.js      # 12 pruebas: deben pasar
```

LAS PRUEBAS ★ SON LAS QUE IMPORTAN

No acuñar sobre el tope; quemar libera espacio; el canje retira exactamente lo aprobado; el contrato no puede tomar más. Esas cuatro son la tesis del laboratorio.

OPENZEPPELIN: COMPONER EXTENSIONES

```
contract FichaUSB is ERC20, ERC20Burnable, ERC20Capped, Ownable {
    constructor(uint256 topeEnTokens)
        ERC20("Ficha USB", "FUSB")
        ERC20Capped(topeEnTokens * 10 ** decimals())
        Ownable(msg.sender)
    {}

    function acunar(address a, uint256 cant) external onlyOwner {
        _mint(a, cant * 10 ** decimals());
    }
    function _update(address from, address to, uint256 v)
        internal override(ERC20, ERC20Capped) { super._update(from, to, v); }
}
```

- ERC20: las seis funciones.
- Burnable: quemar los propios.
- Capped: tope de emisión.
- Ownable: solo el dueño acuña.
- Lo único propio: acunar y el _update que compone las extensiones.

El _update override es obligatorio en OpenZeppelin v5: al mezclar ERC20 y Capped hay que decir cuál gana. Sin él, no compila — a propósito, para entender que las extensiones se componen aquí.

LOS CUATRO TODO, EN ORDEN

01	TODO 1 · EL TOPE	En el constructor: ERC20Capped(topeEnTokens * 10 ** decimals()). Sin convertir, el tope es una fracción invisible.
02	TODO 2 · ACUNAR	_mint(a, cantidadEnTokens * 10 ** decimals()). No revisar el tope aquí: lo hace Capped en _update.
03	TODO 3 · _UPDATE	Una línea: super._update(from, to, value). Mientras esté vacío, NINGÚN saldo se mueve.
04	TODO 4 · CANJEAR	transferFrom(msg.sender, tesoreria, precio), revert si false, sumar el premio y emitir Canjeado (ver A.4).

⚠ EL ORDEN IMPORTA

Empiecen por el TODO 3: sin él casi todas las pruebas fallan, y los otros tres TODO parecen rotos aunque estén bien.

QUÉ PASA EXACTAMENTE EN CANJEAR()

```
function canjear() external {
    // el Canje es msg.sender para la ficha: es el GASTADOR
    bool ok = ficha.transferFrom(msg.sender, tesoreria, precio);
    if (!ok) revert TransferenciaFallida();

    premiosDe[msg.sender] += 1;
    emit Canjeado(msg.sender, precio);
}
```

- Ana llama canjear(): para el Canje, msg.sender = Ana.
- El Canje llama a la ficha: para la ficha, msg.sender = el Canje.
- La ficha revisa allowance[Ana][Canje] ≥ 25, la descuenta y mueve 25 de Ana a la tesorería.
- Si no alcanza, revierte TODO: no hay premio sin pago.

⚠ **EL ERROR TÍPICO DE QUIEN LO PROGRAMA**

Escribir ficha.transferFrom(tesoreria, msg.sender, precio) —al revés—, o usar ficha.transfer(tesoreria, precio): eso intentaría mover los tokens DEL CANJE, que no tiene ninguno. El «de» de transferFrom es el usuario; quien gasta la autorización es el contrato que hace la llamada. Tampoco hace falta un require sobre la allowance: transferFrom ya revierte con ERC20InsufficientAllowance.

SALIDA ESPERADA, AL EMPEZAR Y AL TERMINAR

```
NPX HARDHAT TEST TEST/S10/FICHAUSB.TEST.JS

# con el andamiaje recién copiado (salida real):
  2 passing
  10 failing          # pasan solo nombre/decimales y "solo el organizador acuña"

# con los cuatro TODO completos:
  12 passing          # esa es la meta

# ensayo del despliegue en la RED LOCAL de Hardhat (sin un peso):
[ FichaCanjeModulo ] successfully deployed
FichaCanjeModulo#FichaUSB - 0x5FbD...0aa3
FichaCanjeModulo#Canje    - 0xe7f1...0512
```

TERMINADO SIGNIFICA DOS COSAS (TODO LOCAL)

(1) 12 pruebas en verde en su máquina. (2) Un informe corto: por qué canjear() no necesita require sobre la allowance, y qué pasa en un segundo canje sin re-aprobar. Nada on-chain.

TRES PREGUNTAS ANTES DE CERRAR

PREGUNTA	RESPUESTA ESPERADA
El token tiene 18 decimales. ¿Qué valor debe recibir approve para autorizar 25 fichas?	25 × 10^{18} = 25000000000000000000 (parseUnits("25", 18)).
Ana aprobó 25 al Canje y luego transfirió todo su saldo a Beto. ¿Funciona canjear()?	No: la allowance sigue en 25, pero Ana no tiene saldo. approve no aparta tokens.
¿Por qué FichaUSB no hereda Pausable?	Porque el freno sería un poder centralizado sin necesidad en este caso.

CÓMO USAR ESTA LÁMINA

Tapar la segunda columna. Una pareja responde cada pregunta en voz alta; otra la corrige. Si la mayoría falla la segunda, repasen la tabla de saldos de la magistral: es el error que más tiempo cuesta en el laboratorio.

B

APROXIMADAMENTE 70 MINUTOS · EN PAREJAS

LAB 2 · DIPLOMA NFT Y EL INTRANSFERIBLE

DiplomaUSB (ERC-721 con tope) y DiplomaSBT, el mismo diploma pero «atado al alma» (ERC-5192). Todo local; el tokenURI es una cadena de prueba.

CUATRO PASOS, EN ESTE ORDEN

01	CONTRATO DIPLOMA (20 MIN)	Completar los TODO de emitir() y totalEmitidos() en DiplomaUSB. Meta: 11 pruebas en verde.
02	EL INTRANSFERIBLE (20)	Reto opcional: los TODO de DiplomaSBT (locked, revocar, el if en _update). 9 pruebas más.
03	LEER EL CONTRATO (15)	En la consola de Hardhat: emitir un diploma con un tokenURI de prueba y leer su dueño y su tokenURI.
04	INFORME (15)	Responder las preguntas que traen los andamiajes (ver la última lámina).

[-] NADA SALE DE LA MÁQUINA

El tokenURI es solo una cadena de prueba: hoy no se sube nada a IPFS. Todo corre en la red local de Hardhat, sin ETH.

EMITIR(): CHECKS, EFFECTS, INTERACTION

CONTRACTS/S11/DIPLOMAUSB.SOL · LA SOLUCIÓN

```
function emitir(address a, string calldata uri)
    external onlyOwner returns (uint256 tokenId)
{
    if (bytes(uri).length == 0) revert URIVacio();          // checks
    if (_siguienteId > tope) revert TopeAlcanzado(tope);
    tokenId = _siguienteId++;                               // effects
    _setTokenURI(tokenId, uri);
    _safeMint(a, tokenId);        // interaction: puede llamar al receptor
    emit DiplomaEmitido(tokenId, a, uri);
}
```

- Copien andamiaje/s11/DiplomaUSB.sol sobre contracts/s11/.
- Al empezar: 4 pruebas pasan y 7 (las ★) fallan.
- Terminado: 11 passing.

```
npx hardhat test test/s11/DiplomaUSB.test.js
```

¿Por qué _safeMint va al final? Si el receptor es un contrato, se le llama; el estado ya debe estar escrito (checks-effects-interactions, de la sesión de seguridad).

UN IF EN _UPDATE BLOQUEA LA TRANSFERENCIA

CONTRACTS/S11/DIPLOMASBT.SOL

```
function _update(address to, uint256 tokenId, address auth)
    internal override returns (address)
{
    address from = _ownerOf(tokenId);
    if (from != address(0) && to != address(0))       // ni mint ni quema
        revert DiplomaIntransferible(tokenId);          // = transferencia
    address anterior = super._update(to, tokenId, auth);
    if (from == address(0)) emit Locked(tokenId);      // ERC-5192
    return anterior;
}
```

- En OpenZeppelin v5 todo movimiento pasa por _update.
- Emitir (de 0) y revocar (a 0) siguen permitidos.
- Reto opcional: andamiaje/s11/DiplomaSBT.sol.

LO QUE MUESTRA LA PRUEBA «BILLETERA PERDIDA»

La universidad revoca el token 1 y emite el 2 a la cuenta nueva. El 1 queda quemado para siempre. ¿Es eso un diploma «inmutable»? ¿Quién tiene el poder real?

SALIDA ESPERADA, AL EMPEZAR Y AL TERMINAR

```
NPX HARDHAT TEST TEST/S11/DIPLOMAUSB.TEST.JS TEST/S11/DIPLOMASBT.TEST.JS

# DiplomaUSB, con el andamiaje recién copiado:
  4 passing
  7 failing           # las ★: tope, uri vacío, emitir, totalEmitidos

# DiplomaUSB, con los TODO completos:
  11 passing

# el intransferible (reto opcional), terminado:
  9 passing           # transferFrom revierte, locked, revocar, reemitir

# los dos archivos juntos:
  20 passing          # esa es la meta
```

TERMINADO SIGNIFICA (TODO LOCAL)

20 pruebas en verde (11 + 9) y un informe con las preguntas respondidas. Nada on-chain: la evidencia es la terminal.

ERRORES COMUNES DEL LABORATORIO

SÍNTOMA	CAUSA Y ARREGLO
No compila: «Derived contract must override function _update»	En DiplomaSBT falta la firma exacta _update(address,uint256,address). Cópienla del andamiaje.
ERC721NonexistentToken al consultar el dueño	Preguntan por un tokenId que no se emitió, o emitir() todavía tiene los TODO sin completar.
La prueba de «no transferible» NO revierte	El if de _update está mal: debe revertir solo cuando from y to son AMBOS distintos de address(0).
TopeAlcanzado al emitir en una prueba	El tope del constructor se fijó más bajo que los diplomas que la prueba intenta emitir.
OwnableUnauthorizedAccount	emitir() o revocar() se llaman desde una cuenta que no es la que desplegó (el owner).
Locked no se emite al emitir	Falta el emit Locked(tokenId) cuando from == address(0) (el último TODO del SBT).

EVIDENCIA DEL LABORATORIO (LOCAL)

- **Captura de la terminal con 12 passing (Lab 1) y 20 passing (Lab 2).**
- Sus archivos completos: FichaUSB.sol, Canje.sol, DiplomaUSB.sol y —si hicieron el reto— DiplomaSBT.sol.
- Un informe corto (media página) que responda las preguntas de los andamiajes.

LAS PREGUNTAS DEL INFORME (VIENEN EN LOS ANDAMIAJES)

(1) ¿Qué evento estándar de ERC-721 aparece en el recibo de emitir(), además de DiplomaEmitido, y con qué dirección de origen? (2) ¿Por qué canjear() NO necesita un require sobre la allowance? (3) ¿Por qué DiplomaUSB no compila sin los override de tokenURI y supportsInterface? (4) En el reto del SBT: tras «billetera perdida», ¿quién conserva el poder real sobre el diploma?

C

ENTREGA Y RÚBRICA

LA ACTIVIDAD: QUÉ SE ENTREGA Y CÓMO SE CALIFICA

Trabajo en grupos, en red local. La prueba en verde es parte de la nota.

QUÉ SE ENTREGA

Una sola entrega por grupo (los mismos del proyecto). Todo corre en la red local de Hardhat: no se necesita ETH, ni red pública, ni IPFS.

EVIDENCIA	DETALLE
Contratos completos	FichaUSB.sol, Canje.sol, DiplomaUSB.sol y DiplomaSBT.sol con los // TODO resueltos.
Capturas de pruebas	Terminal con 12 passing (Lab 1) y 20 passing (Lab 2) a la vista.
Informe corto	Las 4 preguntas de reflexión respondidas con sus palabras (3–5 líneas cada una).

LO ESENCIAL

La evidencia es LOCAL: no se pide hash de transacción ni contrato verificado, porque no se despliega a ninguna red pública. Basta la terminal en verde y el informe.

CÓMO SE CALIFICA · 100 PUNTOS

Cuatro componentes. La parte automática es la prueba en verde (pasa o no pasa); la de argumento mide que expliquen el código con sus palabras.

COMPONENTE	PTS	REPARTO INTERNO
Lab 1 · Token (FichaUSB + Canje)	30	Pruebas en verde 22 (las 12) · explican los 4 TODO 8
Lab 2 · Diploma NFT (DiplomaUSB)	30	Pruebas en verde 22 (las 11) · C-E-I y los override 8
Lab 2b · SBT intransferible (DiplomaSBT)	20	Pruebas en verde 14 (las 9) · el if en _update y su trade-off 6
Informe y reflexión	20	Las 4 preguntas de reflexión

58 DE 100 SON AUTOMÁTICOS

La parte de pruebas (22 + 22 + 14) la verifican ustedes mismos con npm run ficha y npm run diploma. Los otros 42 puntos miden que entiendan lo que escribieron.

PREGUNTAS DE REFLEXIÓN

Valen 20 puntos. Se responden mirando SU código y SU proyecto, no en abstracto.

1. ¿Qué evento estándar de ERC-721 emite emitir() además de DiplomaEmitido, y con qué dirección de origen?
1. ¿Por qué canjear() NO necesita un require sobre la allowance? (pruébenlo sin aprobar y miren el error).
1. ¿Por qué el contrato no compila sin los override de tokenURI y supportsInterface?
1. ¿Cuál de las tres familias (fungible / NFT / SBT) le aplicaría a su propio proyecto, y en qué función?

RESPUESTAS GENÉRICAS = 0

La respuesta que valdría igual para cualquier curso —la que produce un asistente sin ver su código— no recibe puntos, aunque esté «bien escrita».

LO QUE SE LLEVAN DE HOY

Lo fungible es una tabla de saldos; lo único, una fila por token. Y los dos salen de OpenZeppelin.

Ayer lo explicamos en el tablero; hoy lo construyeron y lo dejaron en verde, sin un peso y sin salir de su máquina.

SIGUIENTE · FRONTEND WEB3 · CONECTAR LA INTERFAZ A LA CADENA